This is the final **System Integration & Workflow Reference** for your repository. It explains *how* the modules talk to each other and provides the "glue" logic that makes them a single system rather than just a collection of scripts.

High-Level Summary: How it Fits Together

Think of **Velites** as a digital assembly line for financial intelligence:

1. **Raw Material (The Scout):** The system wakes up and scrapes "raw ore" (ArXiv papers, Patent grants, News headlines). It dumps this into a temporary holding queue.
2. **Processing (The Mapper):** The raw text is passed through your "Knowledge Graph." The system tags vague terms like "Blackwell" with specific tickers like NVDA (and its suppliers like TSM). It filters out noise—if no ticker is found, the data is discarded.
3. **Refinement (The Analyst):** The tagged data goes to the AI.
    - The **Innovation Agent** reads the technical paper to see if it's a breakthrough.
    - The **Sentiment Agent** reads the news to see if the market is already hyped.
    - The **Confluence Engine** compares the two. If Science says "Go" and News says "Quiet," it prints a ticket.
4. **Delivery (The Courier):** The system packages this "Buy Ticket" into a clean JSON file and sends it to your **Homeguard** execution bot.

Velites System Integration Reference

Version: 1.0
Repository: Velites (Research Sidecar)
Target System: Homeguard (Execution Engine)

---

1. The Integration Workflow (The "Loop")

This system is designed to run as a **DAG (Directed Acyclic Graph)** workflow, triggered periodically (e.g., every 4 hours).

Step 1: Ingestion & Normalization

- **Trigger:** Cron Job or Event Loop.
- **Action:**
    1. Scout fetches last 4 hours of ArXiv papers (cs.AI).
    2. Scout fetches last 4 hours of News for the "Watchlist" (from knowledge_graph.json).
- **Data Payload (Internal):**
  JSON
  ```
  {
    "id": "arxiv_2601.12345",
    "type": "PAPER",
    "text": "Abstract: We present a novel lithography technique using...",
  ```

```
  "timestamp": "2026-01-16T10:00:00Z"
}
```

Step 2: Entity Resolution (The Filter)

- **Action:** Mapper receives the payload.
- **Logic:**
  1. Scans text for keywords from product_map (e.g., "Lithography", "EUV").
  2. Resolves to Ticker: ASML.
  3. **Expansion:** Looks up dependencies. "If ASML is mentioned, also check LRCX (competitor) and INTC (customer)."
- **Enriched Payload:**

```
JSON
{
  "id": "arxiv_2601.12345",
  "primary_ticker": "ASML",
  "related_tickers": ["LRCX", "INTC"],
  "sector": "semiconductor_equipment"
}
```

Step 3: Signal Generation (The Brain)

- **Action:** Analyst takes the Enriched Payload.
- **Parallel Processes:**
  - **Process A (Innovation):** Sends abstract to LLM.
    - *Result:* Score: 0.9 (Breakthrough).
  - **Process B (Sentiment):** Checks recent News Sentiment for ASML.
    - *Result:* Score: 0.1 (Neutral/Quiet).
- **Decision Logic (Confluence):**
  - IF Innovation > 0.7 AND Sentiment > -0.5 THEN SIGNAL = TRUE.

Step 4: Liquidity Check & Dispatch (The Handover)

- **Action:** Courier prepares the message for Homeguard.
- **Logic:**
  1. Checks Ticker_Normalization. Is ASML tradeable? Yes (NASDAQ).
  2. Checks Market_Feeder. Is Spread < 1%? Yes.
- **Final Output (Sent to Homeguard via Webhook/File):**

```
JSON
{
  "signal_id": "velites_001",
  "action": "BUY_LONG",
  "ticker": "ASML",
```

```
  "confidence": 0.9,
  "reasoning": "ArXiv paper suggests new moat expansion. Market sentiment is quiet.",
  "valid_until": "2026-01-17T10:00:00Z"
}
```

---

2. Module Interface Contracts

These are the specific functions you must implement to make the modules "talk."

A. Scout $\leftrightarrow$ Mapper

- **Function:** mapper.resolve_text(text_string) -> List[TickerObject]
- **Input:** "New Blackwell delay rumors surfacing."
- **Output:** [{ticker: "NVDA", role: "Direct"}, {ticker: "TSM", role: "Supplier"}]

B. Mapper $\leftrightarrow$ Analyst

- **Function:** analyst.grade_innovation(text, ticker_context) -> float
- **Input:** (Paper Abstract, "NVDA is a GPU Designer")
- **Output:** 0.85

C. Analyst $\leftrightarrow$ Courier

- **Function:** courier.validate_liquidity(ticker) -> bool
- **Input:** SSNLF
- **Logic:** Checks if it's an OTC stock with low volume.
- **Output:** False (Do not trade).

---

3. Failure Modes & Recovery

- **LLM Hallucination:**
  - *Risk:* LLM thinks "Silicon Valley" (TV show) is "Silicon" (Material).
  - *Fix:* Mapper keywords must be strict. Only send text to LLM if it matches specific technical terms (e.g., "Wafer", "GAAFET").
- **Zero Liquidity:**
  - *Risk:* System tries to buy a Korean stock on a US holiday.
  - *Fix:* Courier checks market status before dispatch. If market closed, queue the signal for Open.

---

4. Directory Structure (Implementation View)

This is how your files should look on Day 1.

Plaintext

```
Velites/
├── data/
│   ├── knowledge_graph_v1_2.json    # The Brain (Your Map)
│   └── watchlist.json               # Auto-generated by the Map
├── modules/
│   ├── scout/
│   │   ├── arxiv_fetcher.py         # The Librarian
│   │   └── news_fetcher.py          # The News Aggregator
│   ├── mapper/
│   │   ├── graph_engine.py          # Entity Resolution Logic
│   │   └── supply_chain.py          # Dependency Traversal
│   ├── analyst/
│   │   ├── llm_agent.py             # OpenAI/Claude Integration
│   │   └── sentiment_engine.py      # FinBERT Logic
│   └── courier/
│       └── dispatcher.py            # JSON Formatter for Homeguard
├── main.py                          # The Event Loop (Orchestrator)
├── requirements.txt
└── README.md
```